

Treino e teste

Por que um modelo é julgado em casos que ele nunca viu

Mateus Levi Simões Fernandes

Aula 4 · 24/09/2026

Aprendizado de Máquina (APMAI) · IASEG

Para que serve um modelo

Um modelo aprende com **casos do passado**, mas é usado em **casos que ainda não existem**:

- o passageiro cujo destino ainda não conhecemos
- o cliente para quem o banco ainda não ligou
- a flor que ninguém classificou

Para que serve um modelo

Um modelo aprende com **casos do passado**, mas é usado em **casos que ainda não existem**:

- o passageiro cujo destino ainda não conhecemos
- o cliente para quem o banco ainda não ligou
- a flor que ninguém classificou

Então a utilidade de um modelo depende de **como ele se sai em casos novos**, não de como ele se saiu nos casos que já viu.

Para que serve um modelo

Um modelo aprende com **casos do passado**, mas é usado em **casos que ainda não existem**:

- o passageiro cujo destino ainda não conhecemos
- o cliente para quem o banco ainda não ligou
- a flor que ninguém classificou

Então a utilidade de um modelo depende de **como ele se sai em casos novos**, não de como ele se saiu nos casos que já viu.

Para testar isso, precisamos **fingir** que alguns casos são novos, deixando uma parte dos dados fora do treinamento.

Uma analogia: lista de exercícios vs. a prova

Se a prova é igual à lista de exercícios

- quem entendeu a matéria vai tirar uma nota boa
- quem decorou as respostas da lista também vai tirar uma nota boa
- ou seja, a nota **não vai separar** um grupo do outro

Uma analogia: lista de exercícios vs. a prova

Se a prova é igual à lista de exercícios

- quem entendeu a matéria vai tirar uma nota boa
- quem decorou as respostas da lista também vai tirar uma nota boa
- ou seja, a nota **não vai separar** um grupo do outro

Prova com questões diferentes da lista

- quem entendeu a matéria continua indo bem
- quem só decorou os exercícios não vai tirar uma nota boa
- a nota **mede o que importa** (quem de fato entendeu a matéria)

Uma analogia: lista de exercícios vs. a prova

Se a prova é igual à lista de exercícios

- quem entendeu a matéria vai tirar uma nota boa
- quem decorou as respostas da lista também vai tirar uma nota boa
- ou seja, a nota **não vai separar** um grupo do outro

Prova com questões diferentes da lista

- quem entendeu a matéria continua indo bem
- quem só decorou os exercícios não vai tirar uma nota boa
- a nota **mede o que importa** (quem de fato entendeu a matéria)

Treino = a lista de exercícios. **Teste** = a prova com questões novas.

O modelo que só decora

Para detalhar isso, podemos criar um modelo que "decora" o treino, funcionando da seguinte forma:

- guarda cada passageiro do treino, com a resposta dele
- se aparece um passageiro idêntico a um que já viu, repete a resposta que tinha no treino
- se não, chuta a classe mais comum ("não sobreviveu")

O modelo que só decora

Para detalhar isso, podemos criar um modelo que "decora" o treino, funcionando da seguinte forma:

- guarda cada passageiro do treino, com a resposta dele
- se aparece um passageiro idêntico a um que já viu, repete a resposta que tinha no treino
- se não, chuta a classe mais comum ("não sobreviveu")

Titanic (714 passageiros)	Treino (499)	Teste (215)
Modelo que decora	99,4%	58,6%
Referência (classe mais comum: sempre "não")	59,3%	59,5%
kNN com $k = 1$	99,4%	63,3%

O modelo que só decora

Para detalhar isso, podemos criar um modelo que "decora" o treino, funcionando da seguinte forma:

- guarda cada passageiro do treino, com a resposta dele
- se aparece um passageiro idêntico a um que já viu, repete a resposta que tinha no treino
- se não, chuta a classe mais comum ("não sobreviveu")

Titanic (714 passageiros)	Treino (499)	Teste (215)
Modelo que decora	99,4%	58,6%
Referência (classe mais comum: sempre "não")	59,3%	59,5%
kNN com $k = 1$	99,4%	63,3%

(Só 22 dos 215 passageiros do teste são idênticos a alguém do treino.)

O modelo que só decora

Para detalhar isso, podemos criar um modelo que "decora" o treino, funcionando da seguinte forma:

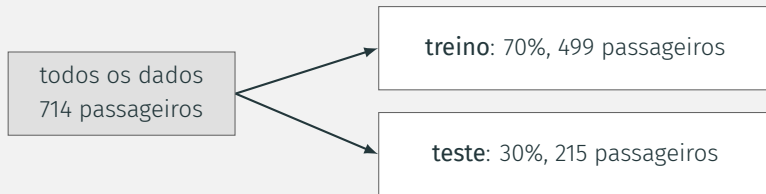
- guarda cada passageiro do treino, com a resposta dele
- se aparece um passageiro idêntico a um que já viu, repete a resposta que tinha no treino
- se não, chuta a classe mais comum ("não sobreviveu")

Titanic (714 passageiros)	Treino (499)	Teste (215)
Modelo que decora	99,4%	58,6%
Referência (classe mais comum: sempre "não")	59,3%	59,5%
kNN com $k = 1$	99,4%	63,3%

(Só 22 dos 215 passageiros do teste são idênticos a alguém do treino.)

Avaliar um modelo pela performance no treino favorece modelos que "decoram".

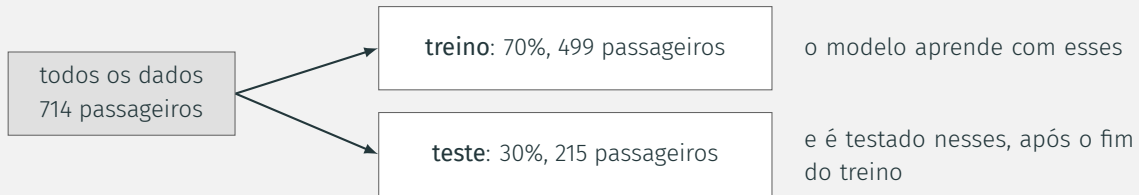
A ideia: separar antes de treinar



o modelo aprende com esses

e é testado nesses, após o fim do treino

A ideia: separar antes de treinar



Por isso existem **duas** medidas de desempenho, e elas respondem perguntas diferentes:

- **no treino:** o quanto o modelo se ajustou aos casos que viu
- **no teste:** a melhor estimativa de como ele vai se sair em casos novos

No código: uma linha, três decisões

```
from sklearn.model_selection import train_test_split  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.3, random_state=42, stratify=y)
```

No código: uma linha, três decisões

```
from sklearn.model_selection import train_test_split  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.3, random_state=42, stratify=y)
```

test_size=0.3 – qual proporção dos dados vamos deixar para o conjunto de teste. Quanto maior, mais confiável é o teste, mas sobram menos dados para o modelo aprender.

No código: uma linha, três decisões

```
from sklearn.model_selection import train_test_split  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.3, random_state=42, stratify=y)
```

test_size=0.3 – qual proporção dos dados vamos deixar para o conjunto de teste. Quanto maior, mais confiável é o teste, mas sobram menos dados para o modelo aprender.

random_state=42 a divisão é aleatória; fixamos o número para poder **reproduzir** a divisão ao rodar o código.

No código: uma linha, três decisões

```
from sklearn.model_selection import train_test_split  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.3, random_state=42, stratify=y)
```

test_size=0.3 – qual proporção dos dados vamos deixar para o conjunto de teste. Quanto maior, mais confiável é o teste, mas sobram menos dados para o modelo aprender.

random_state=42 a divisão é aleatória; fixamos o número para poder **reproduzir** a divisão ao rodar o código.

stratify=y mantém a proporção de cada classe igual no treino e no teste.

O que o `stratify` garante

No Titanic (714 passageiros), 290 sobreviveram: 40,6%. Quanto disso cai em cada parte?

O que o `stratify` garante

No Titanic (714 passageiros), 290 sobreviveram: 40,6%. Quanto disso cai em cada parte?

	Sobreviventes no treino	Sobreviventes no teste
Com <code>stratify=y</code>	40,7%	40,5%
Sem <code>stratify</code> , <code>random_state=42</code>	40,3%	41,4%
Sem <code>stratify</code> , 20 sorteios	—	de 36,3% a 46,0%

O que o `stratify` garante

No Titanic (714 passageiros), **290 sobreviveram: 40,6%**. Quanto disso cai em cada parte?

	Sobreviventes no treino	Sobreviventes no teste
Com <code>stratify=y</code>	40,7%	40,5%
Sem <code>stratify</code> , <code>random_state=42</code>	40,3%	41,4%
Sem <code>stratify</code> , 20 sorteios	—	de 36,3% a 46,0%

Sem `stratify`, o conjunto de teste pode sair com uma proporção de classes diferente da original. Isso faz com que a nota do teste seja enviesada.

O que o `stratify` garante

No Titanic (714 passageiros), **290 sobreviveram: 40,6%**. Quanto disso cai em cada parte?

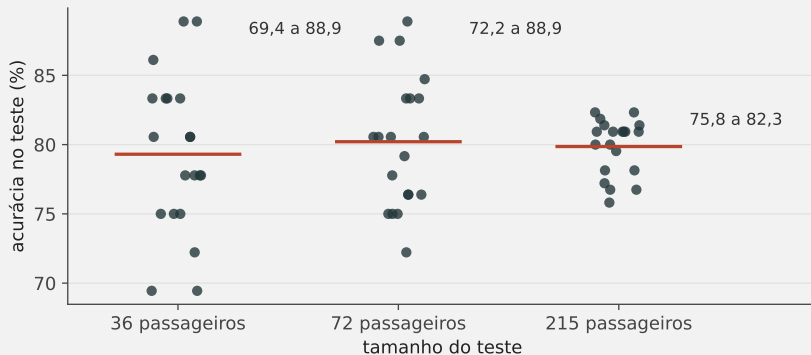
	Sobreviventes no treino	Sobreviventes no teste
Com <code>stratify=y</code>	40,7%	40,5%
Sem <code>stratify</code> , <code>random_state=42</code>	40,3%	41,4%
Sem <code>stratify</code> , 20 sorteios	—	de 36,3% a 46,0%

Sem `stratify`, o conjunto de teste pode sair com uma proporção de classes diferente da original. Isso faz com que a nota do teste seja enviesada.

Padrão: para problemas de classificação, usamos `stratify=y`.

A nota do teste depende da divisão feita

O mesmo modelo (uma árvore com profundidade 3) no Titanic, com **20 sorteios** diferentes da divisão:



Então: quanto guardar para o teste?

Teste pequeno

- Sobra mais para aprender
- Desempenho no teste varia mais

Teste grande

- Menos dados pro modelo aprender
- Desempenho no teste fica mais estável

Então: quanto guardar para o teste?

Teste pequeno

- Sobra mais para aprender
- Desempenho no teste varia mais

Teste grande

- Menos dados pro modelo aprender
- Desempenho no teste fica mais estável

Padrão: guardar de 20% a 30%.

Antes: por que escalar os dados?

O kNN decide pela **distância**: soma as diferenças entre dois passageiros, coluna por coluna.

Antes: por que escalar os dados?

O kNN decide pela **distância**: soma as diferenças entre dois passageiros, coluna por coluna.

Mas cada coluna tem sua própria faixa:

- tarifa: de 0 a 512
- idade: de 0 a 80
- sexo: 0 ou 1

Antes: por que escalar os dados?

O kNN decide pela **distância**: soma as diferenças entre dois passageiros, coluna por coluna.

Mas cada coluna tem sua própria faixa:

- tarifa: de 0 a 512
- idade: de 0 a 80
- sexo: 0 ou 1

Uma passageira do teste: **mulher, 1ª classe, 31 anos.**

Antes: por que escalar os dados?

O kNN decide pela **distância**: soma as diferenças entre dois passageiros, coluna por coluna.

Mas cada coluna tem sua própria faixa:

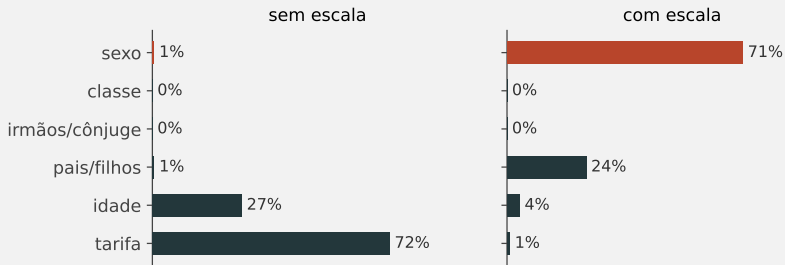
- tarifa: de 0 a 512
- idade: de 0 a 80
- sexo: 0 ou 1

Uma passageira do teste: **mulher, 1ª classe, 31 anos**.

O vizinho mais próximo dela no treino é **um homem** (1ª classe, 38 anos).

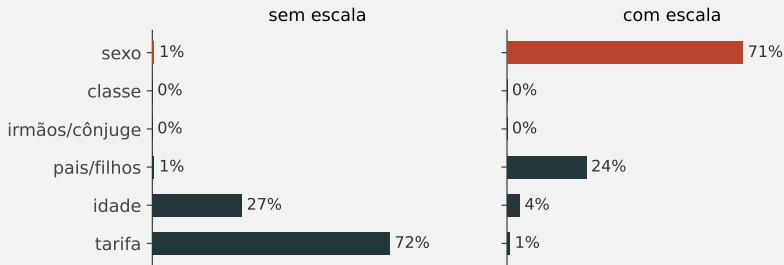
Escalar: todas as colunas na mesma régua

quanto cada coluna pesa na distância entre ela e o vizinho



Escalar: todas as colunas na mesma régua

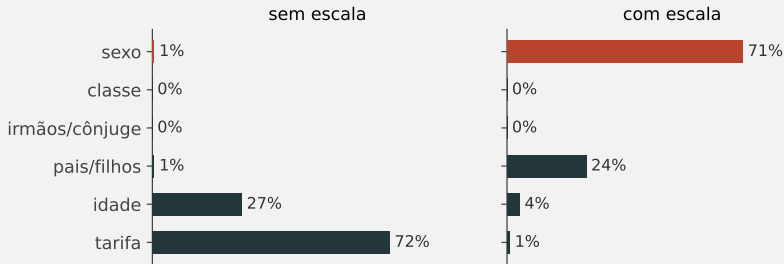
quanto cada coluna pesa na distância entre ela e o vizinho



Escalar = medir cada diferença pelo **quanto aquela coluna costuma variar**.

Escalar: todas as colunas na mesma régua

quanto cada coluna pesa na distância entre ela e o vizinho



Escalar = medir cada diferença pelo **quanto aquela coluna costuma variar**.

- vizinho mais próximo do sexo oposto: 65 de 215 → **0 de 215**
- kNN ($k = 5$): 140 de 215 acertos → **167 de 215**

Padrão: `StandardScaler` (kNN e regressão logística precisam; a árvore, não).

Como dá errado (1): o teste vaza para o treino

O teste só funciona se **nada** dele entrar no treino. Exemplos de vazamento:

- aprender a escala com **todos** os dados
- preencher valores faltando com a média da base **inteira**
- escolher colunas olhando o **teste**

Como dá errado (1): o teste vaza para o treino

O teste só funciona se **nada** dele entrar no treino. Exemplos de vazamento:

- aprender a escala com **todos** os dados
- preencher valores faltando com a média da base **inteira**
- escolher colunas olhando o **teste**

Padrão: dividir primeiro; tudo que aprende com os dados aprende **só no treino**.

```
scaler.fit(X_train)           # aprende a escala só no treino
X_train_s = scaler.transform(X_train)
X_test_s  = scaler.transform(X_test)  # aplica a mesma escala no teste
```

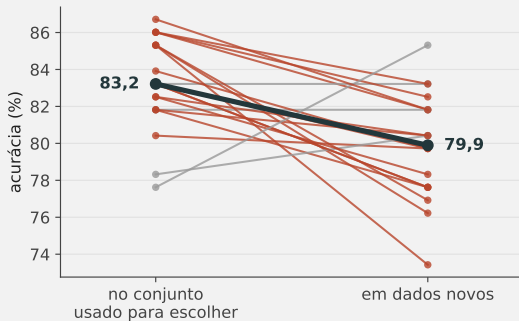
Como dá errado (2): escolher olhando o teste

Escolher o k do kNN (de 1 a 30) pela nota num conjunto guardado, e medir o escolhido em **dados novos**. 20 sorteios.

Como dá errado (2): escolher olhando o teste

Escolher o k do kNN (de 1 a 30) pela nota num conjunto guardado, e medir o escolhido em **dados novos**. 20 sorteios.

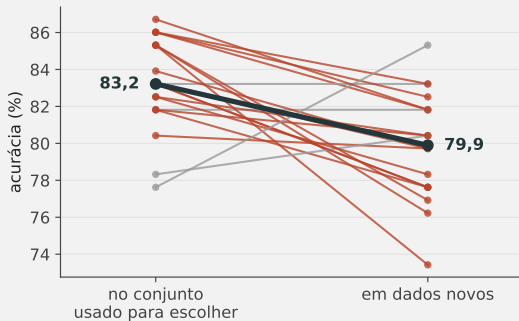
- na escolha: **83,2%**
- em dados novos: **79,9%**
- otimista em **16 de 20**



Como dá errado (2): escolher olhando o teste

Escolher o k do kNN (de 1 a 30) pela nota num conjunto guardado, e medir o escolhido em **dados novos**. 20 sorteios.

- na escolha: **83,2%**
- em dados novos: **79,9%**
- otimista em **16 de 20**



Padrão: o conjunto que ajudou a escolher não serve mais para julgar. **Aula 5:** validação cruzada.

Na prática

1. Dividir **antes de tudo**
2. Classificação: `stratify=y`
3. Fixar o `random_state`
4. Escala e modelo aprendem **só no treino**
5. Mostrar **treino, teste e referência** lado a lado
6. Usar o teste **uma vez**, no fim

Na prática

1. Dividir **antes de tudo**
2. Classificação: **stratify=y**
3. Fixar o **random_state**
4. Escala e modelo aprendem **só no treino**
5. Mostrar **treino, teste e referência** lado a lado
6. Usar o teste **uma vez**, no fim

E o que treino e teste dizem **juntos**? Próxima parte.

Perguntas da arguição:

- **7.** Como vocês sabem que o modelo não decorou os dados de treino, e como sabem que ele aprendeu alguma coisa?
- **11.** Se refizessem a divisão treino/teste, o resultado mudaria muito? Como vocês sabem?

Para o projeto

Perguntas da arguição:

- **7.** Como vocês sabem que o modelo não decorou os dados de treino, e como sabem que ele aprendeu alguma coisa?
- **11.** Se refizessem a divisão treino/teste, o resultado mudaria muito? Como vocês sabem?

No relatório:

- **Seção 3:** treino, teste e referência, lado a lado
- **Seção 5:** o quanto o resultado depende da divisão